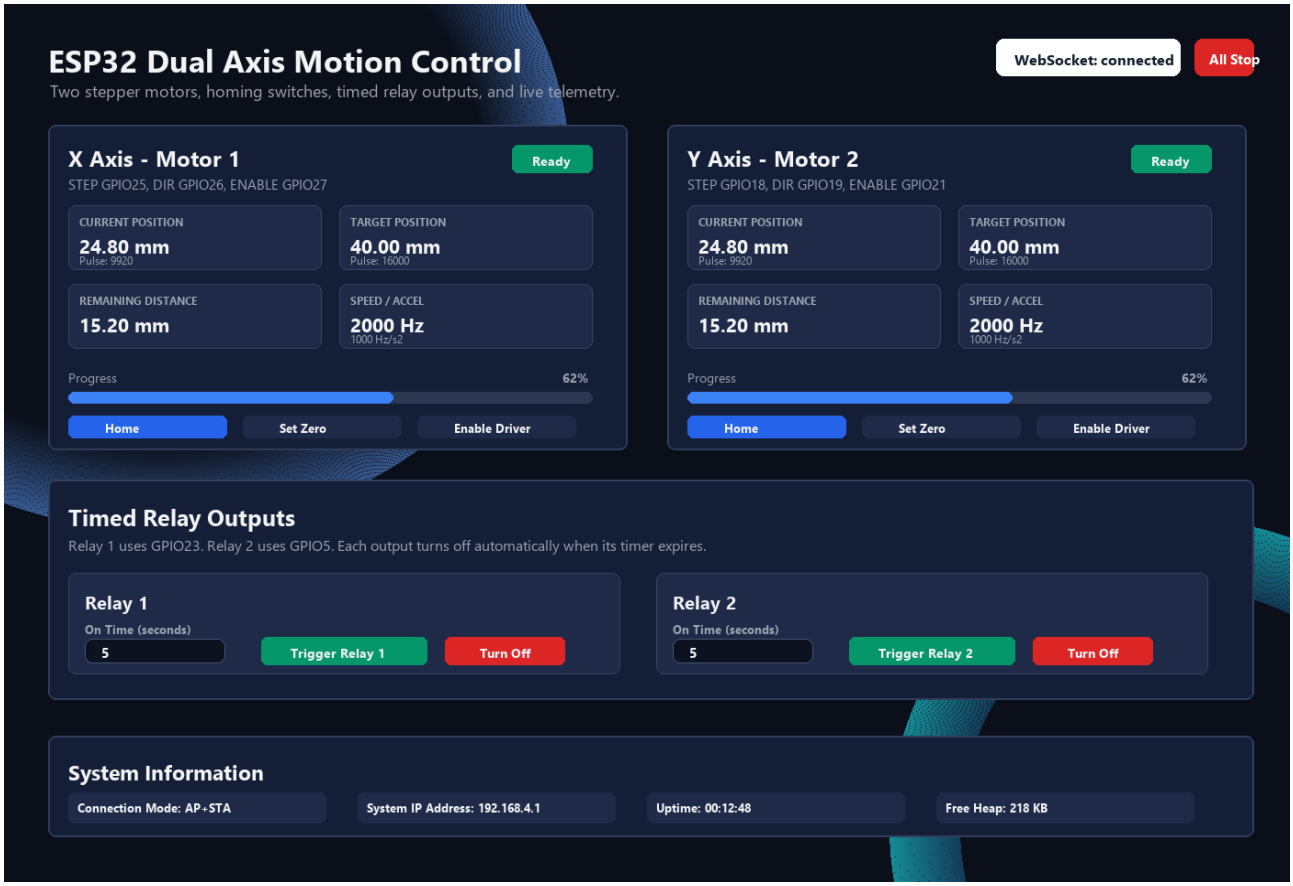
ESP32 Dual Axis Motion Control Walkthrough And Wiring Guide

This guide explains how to build, wire, upload, configure, and test the ESP32 firmware for two stepper motors driven by DRV8825 modules. It also covers the two timed relay outputs controlled from the embedded web dashboard.



1. System Overview

The ESP32 runs an access point named ESP32-MOTOR and serves a browser dashboard from flash memory. The dashboard uses WebSocket telemetry for live motor position, target position, limit switch state, driver enable state, relay state, remaining relay time, uptime, heap, and system log messages.

- Motor 1 controls the X axis through STEP GPIO25, DIR GPIO26, ENABLE GPIO27, and HOME GPIO32.
- Motor 2 controls the Y axis through STEP GPIO18, DIR GPIO19, ENABLE GPIO21, and HOME GPIO33.
- Relay 1 is controlled by GPIO23. Relay 2 is controlled by GPIO5.
- Motor settings, Wi-Fi credentials, and last known motor positions are stored in ESP32 NVS Preferences.

2. Required Hardware

Item	Quantity	Notes
ESP32 DevKit V1 or compatible ESP32 board	1	3.3 V GPIO logic.
DRV8825 stepper driver module	2	One driver per stepper motor.
Stepper motor	2	Set current limit on each DRV8825 before use.
Normally open or normally closed home switch	2	Firmware expects active-low switches using INPUT_PULLUP.
Relay module	2	Use a 3.3 V compatible input module or add a transistor/optocoupler driver.
Motor power supply	1	Sized for motor voltage and current.
5 V or 3.3 V relay supply	1	Depends on relay module type.

3. ESP32 Pin Assignment

Function	ESP32 GPIO	Connect To
Motor 1 STEP	GPIO25	DRV8825 Motor 1 STEP
Motor 1 DIR	GPIO26	DRV8825 Motor 1 DIR
Motor 1 ENABLE	GPIO27	DRV8825 Motor 1 ENABLE
Motor 1 HOME	GPIO32	Home switch signal, other switch terminal to GND
Motor 1 M0	GPIO14	DRV8825 M0
Motor 1 M1	GPIO13	DRV8825 M1
Motor 1 M2	GPIO4	DRV8825 M2
Motor 2 STEP	GPIO18	DRV8825 Motor 2 STEP
Motor 2 DIR	GPIO19	DRV8825 Motor 2 DIR
Motor 2 ENABLE	GPIO21	DRV8825 Motor 2 ENABLE
Motor 2 HOME	GPIO33	Home switch signal, other switch terminal to GND
Motor 2 M0	GPIO16	DRV8825 M0
Motor 2 M1	GPIO17	DRV8825 M1
Motor 2 M2	GPIO22	DRV8825 M2
Relay 1 signal	GPIO23	Relay module IN1 or transistor driver input
Relay 2 signal	GPIO5	Relay module IN2 or transistor driver input

GPIO5 is usable as a relay output on common ESP32 DevKit boards, but it is also a boot strapping pin on ESP32. Make sure the relay module input does not force this pin into the wrong state during reset. If your relay board pulls the pin strongly at boot, move RELAY2_PIN in Config.h to another safe free output pin available on your board.

4. DRV8825 Wiring Notes

- Connect ESP32 GND, both DRV8825 GND pins, relay module GND, and power supply signal ground together.
- Connect DRV8825 VMOT and GND to the motor supply. Add the recommended electrolytic capacitor close to each driver module.
- Connect each stepper motor coil pair to A1/A2 and B1/B2. If direction is reversed, swap one coil pair or invert direction in wiring.
- Connect SLEEP and RESET together and pull them high according to the DRV8825 carrier board recommendations.
- ENABLE is active-low in the firmware: LOW enables the driver, HIGH disables it.
- Set the DRV8825 current limit before connecting the motors for long runs.

5. Home Switch Wiring

The firmware configures each home input as INPUT_PULLUP and treats LOW as active. Wire one side of each switch to the ESP32 home GPIO and the other side to GND. When the switch closes, the input goes LOW and homing stops.

6. Relay Wiring

- Relay 1 signal input connects to GPIO23.
- Relay 2 signal input connects to GPIO5.
- The default firmware assumes active-high relay inputs. The output pin goes HIGH while the relay timer is active and LOW when the timer expires.
- For active-low relay modules, change RELAY_ACTIVE_HIGH in Config.h from true to false.
- Do not drive relay coils directly from ESP32 GPIO pins. Use a relay module with driver circuitry, or add a transistor, flyback diode, and suitable supply.
- Use the relay contact side according to the load requirement: COM and NO for normally-off loads, COM and NC for normally-on loads.

7. Arduino IDE Setup

- Install the ESP32 board package in Arduino IDE using Espressif's Boards Manager package.
- Select an ESP32 Dev Module compatible board profile.
- Install the required libraries: FastAccelStepper, ESPAsyncWebServer, AsyncTCP, ArduinoJson, and Preferences support from the ESP32 core.
- Open main/main.ino from this project folder.
- Confirm the pin assignments in Config.h match your wiring.
- Compile and upload the sketch to the ESP32.

8. First Boot And Browser Access

- After upload, reset the ESP32.
- Connect your computer or phone to the Wi-Fi network ESP32-MOTOR.
- Use password 12345678.
- Open http://192.168.4.1 in a browser.
- The WebSocket status should show connected, and the dashboard should start updating live telemetry.

9. Station Wi-Fi Mode

The device always keeps its access point available. If you save local Wi-Fi credentials from the Wi-Fi Settings tab and restart the ESP32, it also tries to join that network. When connected, the dashboard shows the station IP address so you can access the same interface from your local network.

10. Motor Configuration

- Open the Motor Settings tab.
- Set speed and acceleration in steps per second and steps per second squared.
- Set software minimum and maximum limits in raw step counts.
- Set step angle and microstep mode to match the motor and DRV8825 M0/M1/M2 wiring.
- Choose Lead Screw or Belt And Pulley and enter the matching mechanical parameters.
- Choose the dashboard unit: pulse, mm, cm, or degree.
- Save settings. The firmware stores them in NVS and applies them immediately.

11. Homing Procedure

Press Home for the axis you want to reference. The motor moves toward the switch, backs off, then approaches slowly for repeatable zeroing. After a successful homing cycle, the current position is set to zero and the Home status becomes Complete.

- Keep a hand near power during the first test.
- If the axis moves away from the home switch, reverse the motor wiring or direction wiring before continuing.
- The firmware includes homing timeouts to prevent endless motion if the switch is not reached.

12. Moving And Jogging

- Enter a target position in the dashboard and press Go for absolute positioning.
- Use the -10 and +10 jog buttons for relative movement in the currently selected unit.
- The Stop button stops one motor. All Stop stops both motors and turns both relays off.
- Soft limits are checked before absolute and jog moves. A command outside the configured range is rejected.

13. Timed Relay Operation

- Open the Dashboard tab.
- Enter the desired on time in seconds for Relay 1 or Relay 2.
- Press Trigger Relay 1 or Trigger Relay 2.
- The selected GPIO turns on immediately and the remaining-time counter starts counting down.
- When the timer reaches zero, the firmware turns the relay output off automatically.
- Press Turn Off to cancel the timer and switch a relay off manually.

14. Safety Checklist

- Test motors at low speed and low current before increasing performance settings.
- Verify all grounds are common before connecting control signals.
- Keep motor power wiring physically separated from switch and signal wiring when possible.
- Use external protection and proper enclosures for relay-controlled AC or high-current loads.
- Do not rely on software alone for machine safety. Add hardware emergency stop and fusing where needed.

15. Troubleshooting

Symptom	Likely Cause	Action
Dashboard does not load	Not connected to ESP32 AP or wrong IP	Join ESP32-MOTOR and open http://192.168.4.1.
WebSocket disconnected	Browser lost connection	Refresh the page or move closer to the ESP32.
Motor does not move	Driver disabled, wiring issue, or current limit too low	Check ENABLE, STEP/DIR wiring, VMOT power, and DRV8825 current limit.
Motor moves the wrong way	Direction wiring or coil orientation reversed	Swap one motor coil pair or swap the DIR logic wiring.
Homing never completes	Switch not wired active-low or axis moving away	Check switch wiring to GND and verify movement direction.
Relay stays on	Active-low module with active-high firmware	Set RELAY_ACTIVE_HIGH to false in Config.h.
ESP32 does not boot with relay connected	Relay module affects GPIO5 at reset	Use a module with high impedance input or move RELAY2_PIN to another safe output.

16. File Map

File	Purpose
main/main.ino	Startup sequence, main loop, watchdog, motor and relay updates.
main/Config.h	GPIO pin assignments, Wi-Fi defaults, unit and drive type definitions.
main/MotorController.*	Stepper movement, homing, limits, unit conversion, driver enable timeout.
main/RelayController.*	Timed relay output control.
main/WebServerManager.*	HTTP routes, WebSocket commands, telemetry JSON.
main/index_html.h	Embedded English web dashboard.